Dyer
Innovation

CLAUDE CERTIFIED ARCHITECT · SCENARIO 5

Claude Code for Continuous Integration

A study guide for prompt-based output control, CLI flag awareness, severity calibration, and validation-retry loops in CI/CD pipelines.

LAST UPDATED

April 27, 2026

THIS GUIDE EMPHASIZES

Prompt vs CLI flags vs CLAUDE.md for output control · Synchronous vs Message Batches API selection · Validation-retry loops: when they work · when they don't · Multi-instance review and the self-review limitation

Contents

01 Scenario Overview and Exam Positioning 3

02 Core Terminology and Definitions 4

2.1 Claude Code CLI Flags 4
2.2 Message Batches API 4
2.3 CLAUDE.md in CI Context 4
2.4 Session Context Isolation 4
2.5 --print Mode vs Interactive Mode 4

03 Key Concept Deep Dives 6

3.1 Controlling Output Format: Prompt vs CLI Flags vs CLAUDE.md 6
3.2 Latency Sensitivity: Synchronous vs Batch API 6
3.3 Prompt Precision: Explicit Criteria vs Vague Instructions 6
3.4 Inline Reasoning for Developer Trust 6
3.5 Validation-Retry Loops: When They Work and When They Don't 6
3.6 The detected_pattern Field for False Positive Analysis 6
3.7 Multi-Instance Review: Why Two Claude Instances Beat Self-Review 6

04 The Prompting Technique Hierarchy for CI 9

05 CI/CD Architecture Patterns 10

5.1 Self-Review Limitation Pattern 10
5.2 Incremental Review Pattern 10
5.3 Trust Erosion Pattern 10
5.4 Attention Dilution Pattern 10

06 Batch API: Technical Constraints 12

01 Scenario Overview and Exam Positioning

The Claude Code for CI/CD scenario places you as the architect of an automated code review and analysis pipeline. You integrate Claude Code into continuous integration workflows:

- Running reviews on pull requests
- Generating test cases
- Producing security audits
- Managing API costs across different workload types

Exam domains drawn from

DOMAIN	WEIGHT	KEY TOPICS TESTED
Domain 3: Claude Code Configuration & Workflows	20%	`-p` flag, `--output-format json`, CLAUDE.md vs prompt, session isolation
Domain 4: Prompt Engineering & Structured Output	20%	Few-shot prompting, explicit criteria, severity calibration, inline reasoning, `tool_use` schemas, validation-retry loops
Domain 5: Context Management & Reliability	15%	Context overload, multi-pass decomposition, incremental review, prior findings, multi-instance review

What the exam expects

- Production-grade architectural decisions, not just feature recall
- Distractors are plausible options engineers commonly reach for before understanding the production implications

How this guide is organized (mirrors how the scenario is tested)

- Terminology first

02 Core Terminology and Definitions

Claude Code CLI Flags

FLAG / OPTION	PURPOSE
`-p` / `--print`	Non-interactive mode. Sends prompt, outputs result to stdout, exits. REQUIRED for CI/CD pipelines. Without it, the job hangs waiting for interactive input.
`--output-format json`	Structures CLI output as JSON with metadata (`session_id` , cost, etc.). Used with `-p` for programmatic parsing.
`--output-format stream-json`	Streaming JSON output for real-time processing of large analyses.
`--output-format text`	Plain text output (default). Human-readable but harder to parse.
`--json-schema`	Constrains the JSON output to match a specific schema. Combined with `--output-format json` for guaranteed structured findings.
`--max-turns`	Limits execution turns. Controls cost and prevents runaway sessions.
`--allowedTools`	Restricts which tools Claude can use. Essential for CI security (e.g., only `Read` , no `Bash`).
`--dangerously-skip-permissions`	Bypasses all permission prompts. Use ONLY in fully isolated CI containers.
`--session-id` / `--resume`	Named sessions for multi-step CI workflows. Retains context across invocations.
`--append-system-prompt`	Adds instructions while preserving Claude Code defaults. Preferred for CI customization.

TRAP

KEY EXAM TRAP – Fabricated CLI Flags

- The exam includes plausible-sounding but nonexistent flags as distractors
- Flags that DO NOT EXIST: `--batch`, `CLAUDE_HEADLESS=true` environment variable
- If an answer references a flag not in the official Claude Code documentation, treat it as fabricated

Message Batches API

The Message Batches API is Anthropic's cost-optimization endpoint for asynchronous workloads:

- 50% cost savings vs synchronous Messages API
- Processing window of up to 24 hours (often faster, but not guaranteed)
- Fire-and-forget: submit, poll for completion, retrieve results
- Supports the same request parameters as synchronous API (including tool definitions)
- Uses `custom_id` fields for correlating outputs back to input requests
- Cannot execute tool calls mid-request (no agentic loop support)

CLAUDE.md in CI Context

In CI/CD, CLAUDE.md serves a specific role: providing project context.

- **Defines:** coding standards, architectural conventions, testing frameworks, review criteria Claude should know about
- **Does NOT control:** output format of specific CI tasks — output formatting belongs in the prompt for each invocation

Session Context Isolation

- The same Claude session that generated code is less effective at reviewing its own changes
- It retains reasoning context that makes it less likely to question its decisions — the self-review limitation
- The fix is an independent review instance that does not see the generator's reasoning
- Mirrors human code review: a different person catches things the author missed

`--print` Mode vs Interactive Mode

- **Interactive mode** (default): launches a conversational session that waits for user input
- `--print` **mode** (`-p`): one-shot execution — send prompt, get response, exit
- Every CI/CD integration must use `-p`
- Directly tested; the most fundamental prerequisite

03 Key Concept Deep Dives

Controlling Output Format: Prompt vs CLI Flags vs CLAUDE.md

Domain: 3 + 4

Three distinct mechanisms for influencing Claude Code output in CI:

MECHANISM	WHAT IT CONTROLS	WHEN TO USE
Prompt instructions	Output format of a specific invocation	When you need structured, parseable output for a particular CI task (e.g., review findings as `[FILE:path] [LINE:n]`)
`--output-format json`	CLI wrapper metadata format	When you need to parse session metadata (`session_id` , cost) programmatically
`--json-schema`	Schema constraint on JSON output	Combined with `--output-format json` to enforce specific output structure
CLAUDE.md	Project context (standards, conventions)	For coding standards, architecture rules, review criteria — NOT for per-task output format

TIP

Core concept

— In `--print` mode, OUTPUT CONTENT is shaped by the PROMPT, not by CLI flags

— `--output-format json` wraps the response in JSON metadata, but the actual review content inside is still controlled by what the prompt asks for

— To get `[FILE:path] [LINE:n] [SEVERITY:level]` structured findings, include that format requirement in the prompt text

— CLAUDE.md provides context (what standards to review against), not format (how to structure output)

Three-layer model

- CLAUDE.md — outermost layer: always-present project context
- Prompt — middle layer: task-specific instructions including output format
- CLI flags — innermost layer: metadata wrapper and execution parameters

Latency Sensitivity: Synchronous vs Batch API

Domain: 3 + 5

WORKFLOW CHARACTERISTIC	API CHOICE	REASONING
Blocking / interactive (someone is waiting)	Synchronous Messages API	Developers can't wait up to 24 hours to merge a PR
Scheduled / background (no one is waiting)	Message Batches API	Overnight reports, weekly audits, nightly test gen — 50% savings
Agentic / multi-turn tool use	Synchronous Messages API	Batch API cannot execute tools mid-request and return results

TIP

Common distractor

— "Batch both with timeout fallback to real-time" — over-engineered

— If a workflow needs real-time responses, use synchronous calls

— Don't build a fallback for something that should never be batched

Prompt Precision: Explicit Criteria vs Vague Instructions

Domain: 4

- When Claude produces both false positives (flagging acceptable patterns) and false negatives (missing real issues), the root cause is almost always vague prompt instructions
- The fix is making criteria specific and targeted

VAGUE INSTRUCTION	PROBLEM	PRECISE REPLACEMENT
"Check that comments are accurate"	Flags TODOs, simple descriptions, anything that looks off	"Flag comments only when their claimed behavior contradicts actual code behavior"
"Review for security issues"	Inconsistent depth, misses some patterns	"Check for: SQL injection, XSS, auth bypass, secrets in code. Flag with file, line, severity."
"Ensure code quality"	Vague — quality means different things	"Flag: functions >50 lines, cyclomatic complexity >10, missing error handling on I/O calls"

TIP

Core concept

- Filtering findings after generation is a symptom-level fix — removes some noise but doesn't help Claude FIND the real issues
- Explicit criteria fix both problems simultaneously:
  - Stop false positives by narrowing what to flag
  - Reduce false negatives by telling Claude exactly what to look for
- **Rule of thumb:** if Claude is flagging wrong things AND missing right things, the prompt is too vague

Inline Reasoning for Developer Trust

Domain: 4 + 5

- When developers report investigating findings is slow, the bottleneck is per-finding triage time
- They must click into each finding to understand whether it's real
- **Fix:** include reasoning and confidence inline with each finding so developers can scan and triage without extra clicks

Watch for stated constraints

- A common scenario adds a constraint like "stakeholders have rejected any approach that filters findings before developer review"
- That eliminates auto-suppression and confidence filtering as options
- Read constraints carefully before evaluating options
- **Without inline reasoning:** "Potential null pointer" — developer must click in to understand
- **With inline reasoning:** "Potential null pointer (HIGH — `user` param nullable per line 42, dereferenced without check line 58)" — developer triages instantly

TIP

Core concept

— Categorization (blocking vs suggestion) helps with PRIORITIZATION — which to look at first

— Inline reasoning helps with INVESTIGATION TIME — how long each one takes to evaluate

— Always match the solution to the specific bottleneck stated in the question

Validation-Retry Loops: When They Work and When They Don't

Domain: 4

- A validation-retry loop sends Claude's failed output back as feedback ("your JSON is missing field X; please re-emit") and asks for a corrected response
- One of the highest-leverage reliability patterns in Domain 4 — but only when the failure type is recoverable

The exam tests whether candidates can distinguish two failure categories.

FAILURE TYPE	RETRY OUTCOME
Format mismatch (wrong date format, wrong units, value placed in wrong field)	RESOLVABLE — Claude has the correct value in context but emitted it incorrectly. The retry tells it specifically what to fix.
Schema violation (missing required field, invalid enum, wrong type)	RESOLVABLE — retry with the validation error guides correction.
Arithmetic error (line items don't sum to declared total)	RESOLVABLE — Claude can recompute when told what failed.
Information absent (data is not in the document Claude was given)	NOT RESOLVABLE — retrying produces another fabricated value or another null. Solve by making the field nullable, or by routing to human review.
Subjective judgment (no objectively correct answer)	NOT RESOLVABLE — no ground truth exists for the model to converge on.

Architectural rule

- Before adding a retry, ask whether the missing or wrong value could in principle be derived from the input
- If yes → retry with specific error feedback
- If no → the right answer is one of:
  - Schema change (nullable field, additional `evidence` field)
  - Human routing
  - Accepting the gap
- NOT more attempts at the same impossible task

TIP

Distractor pattern

— Questions describing high failure rates often offer "increase the retry budget" as a tempting option

— When the underlying failure is information-absent, additional retries waste tokens and produce confidently fabricated values

— That outcome is worse than an honest null

The `detected_pattern` Field for False Positive Analysis

Domain 4

The pattern

- When the extractor flags something as a problem, also have it return WHICH pattern it matched
- This single schema addition lets you quickly verify whether each flag is a true positive or a false positive of pattern matching
- Gives you data to systematically improve the prompt over time

How to apply it

- Add a `detected_pattern` field to every finding's structured output
- The value names the heuristic that fired (`"nested-null-check"`, `"unhandled-promise-rejection"`, `"magic-number-in-config"`)
- When developers dismiss findings, aggregate dismissals by `detected_pattern` to discover which patterns consistently produce noise

```
finding:
  type: object
  required: [file, line, severity, message, detected_pattern]
  properties:
    file: {type: string}
    line: {type: integer}
    severity: {enum: [critical, warning, info]}
    message: {type: string}
    detected_pattern: {type: string}    # the heuristic that fired
    suggested_fix: {type: [string, "null"]}
```

Why it matters

- Without `detected_pattern`, dismissals are opaque — you know developers ignored 45 findings last week, but not which categories
- With it, you discover (for example) that `magic-number` findings are dismissed 80% of the time in config files while `unhandled-promise` findings are dismissed only 5% of the time
- You stop flagging magic numbers in config paths and the trust problem disappears, without sacrificing the high-value category
- This is the data-driven mechanism behind the trust-erosion fix in Section 5.3

Multi-Instance Review: Why Two Claude Instances Beat Self-Review

Domain: 5

Why self-review fails

- When the same Claude session that generated content reviews it, the review tends to defend the prior decision
- The model retains the reasoning context that produced the output

Why a second instance works

- A SECOND independent Claude instance gets the original input plus the first instance's output, but NO access to the first's reasoning
- It catches more issues because it evaluates the artifact on its own merits

The two-instance pattern:

1. **Generator instance:** receives the input (code, document, schema), produces the artifact (refactored code, summary, extracted data).
2. **Reviewer instance:** receives the original input AND the generator's output, but starts from a fresh context. Asked to critique the output against the input.

Where it applies (anywhere generation and review are both valuable)

- Code review of generated patches
- Fact-checking summarized documents
- Validating extracted data against source
- Second-opinion analysis of architectural recommendations

Why "self-review with extended thinking" doesn't substitute

- More thinking from the same context amplifies the same reasoning rather than challenging it
- The independence of context — not the depth of analysis — is what produces the second perspective
- Mirrors human code review: a different reviewer catches things the author missed precisely because they did not live through the decisions that produced the code

TIP

Cost note

— Two-instance review roughly doubles per-artifact API cost

— Use it where the cost of a missed defect is high (security-critical code, customer-facing summaries, regulated data extraction)

— NOT as a default for every CI run

04 The Prompting Technique Hierarchy for CI

The exam repeatedly tests selection of the right prompting technique for the right problem. The decision hierarchy, ordered from simplest to most complex:

Level 1 — Explicit Criteria

- **Solves:** Claude flags wrong things / misses right things (both FP and FN)
- **How:** replace vague instructions with specific, targeted criteria defining exactly what to flag
- **Example:** "Flag comments only when claimed behavior contradicts actual code" instead of "check comments are accurate"

EXAMPLE

Level 2 – Few-Shot Examples

— **Solves:** Claude understands WHAT to do but output FORMAT is inconsistent

— **How:** provide 2–4 concrete input/output examples showing the exact structure

— Few-shot examples beat more instructions when instructions alone produce variable results

— **Use for:** output formatting consistency, severity calibration, actionable feedback structure

Level 3 — Context Provision

- **Solves:** Claude produces duplicates or misses existing work
- **How:** include relevant context (existing tests, prior findings, related files) in the prompt
- **Example:** include existing test file so Claude doesn't suggest tests already covered

Level 4 — Multi-Pass Decomposition

- **Solves:** Claude produces inconsistent depth across many files (attention dilution)
- **How:** split into per-file analysis passes + separate cross-file integration pass
- **Needed for:** large PRs (10+ files) with inconsistent feedback quality, contradictory findings, or missed bugs

RULE

Exam decision rule

- Problem is WHAT Claude looks for → fix criteria (Level 1)
- Problem is HOW Claude formats output → add examples (Level 2)
- Problem is Claude lacking information → add context (Level 3)
- Problem is inconsistent depth across many items → decompose (Level 4)

05 CI/CD Architecture Patterns

Self-Review Limitation Pattern

- The same Claude session that generated code retains its reasoning context
- When asked to review its own output, it re-traverses the same logic path and reaches the same conclusion
- A second, independent instance without the generator's reasoning evaluates the code on its own merits
- (Section 3.7 details the two-instance pattern)

Pattern

- Generator Instance → produces code → commits changes → Reviewer Instance (no access to generator reasoning) → reviews code independently

Why self-review fails

- Extended thinking amplifies the same reasoning
- Self-review instructions ask the same process to reconsider conclusions it already dismissed
- More context doesn't fix a judgment issue
- Only a fresh perspective introduces genuinely independent evaluation

Incremental Review Pattern

- When a review runs again after new commits, without prior context it produces duplicate findings on already-fixed code
- **Fix:** include prior review findings in the prompt and instruct Claude to only report new or still-unaddressed issues

TRAP

Anti-patterns

— Post-processing filters (brittle text matching)
— Scope restriction to latest files only (misses cross-file impacts)
— Skipping intermediate reviews (loses feedback loop)

Trust Erosion Pattern

- High false positive rates in some finding categories undermine trust across ALL categories
- Developers start ignoring everything, including accurate findings
- **Fix:** disable noisy categories immediately, run only high-precision categories to rebuild trust, then re-enable improved categories once they meet quality thresholds

RULE

Key principle

— In automated review systems, precision (low false positives) matters more than recall for developer trust
— Remove the noise source immediately rather than gradually improving it while trust continues to erode

Attention Dilution Pattern

- When a single-pass review of many files produces inconsistent depth, contradictory feedback, and missed bugs, the root cause is attention dilution
- Too much content for consistent analysis in one pass

Solution architecture

- Pass 1 (per-file): analyze each file individually for local issues — consistent depth guaranteed
- Pass 2 (integration): separate pass examining cross-file data flow, interface consistency

TIP

Common distractors

06 Batch API: Technical Constraints

The Message Batches API has one critical technical constraint beyond latency: it cannot support agentic tool-use loops.

How an agentic tool-use loop works (and why batch can't do it)

1. Claude analyzes code
2. Decides it needs more context
3. Calls a tool (e.g., "get file contents")
4. Your code executes the tool, returns the result
5. Claude continues analysis

This requires real-time back-and-forth.

Why Batch breaks this

- Batches API is fire-and-forget: submit, poll later
- No mechanism to intercept a tool call mid-request, execute it, and return the result for Claude to continue
- The request either completes without tool execution or it doesn't work

BATCH API CAPABILITY	STATUS
Tool definitions in request parameters	Supported (same as sync API)
Request correlation via `custom_id`	Supported
Mid-request tool execution and return	NOT supported — fundamental blocker
Processing window	Up to 24 hours
Cost savings	50% compared to synchronous

TIP

Exam tip – Batch API distractor patterns

- "Batch API doesn't support tool definitions" → FALSE
- "No request correlation IDs" → FALSE, `custom_id` handles this
- "Latency too slow" → PARTIALLY TRUE but not the PRIMARY constraint for agentic workflows
- The primary constraint for agentic loops is always: cannot execute tools mid-request

07 Quick Reference: Question Pattern Recognition

IF THE QUESTION SAYS...	THE ANSWER IS USUALLY...
"Output is inconsistent / vague / not actionable"	Few-shot examples (2–4 concrete input/output examples)
"Claude flags wrong things AND misses right things"	Explicit criteria (replace vague instructions with specific conditions)
"Developers waiting / blocks merging"	Synchronous API (never batch for blocking workflows)
"Runs overnight / scheduled / weekly"	Message Batches API (50% savings, latency doesn't matter)
"Pipeline hangs waiting for input"	`-p` flag (`--print` mode for non-interactive execution)
"Duplicate findings after new commits"	Include prior findings in context
"Duplicate test suggestions"	Include existing test file in context
"Inconsistent depth across many files"	Multi-pass decomposition (per-file + integration)
"Same instance reviews its own code"	Independent review instance (two-instance pattern)
"Developers ignoring all findings"	Disable noisy categories, keep high-precision ones
"High false positive rates"	Precision > recall; remove noise sources immediately
"Investigation time is the bottleneck"	Inline reasoning with confidence per finding
"Severity ratings are inconsistent"	Explicit criteria with concrete examples per severity level
"Validation fails because data isn't in the source"	Make the field nullable; do not retry
"Validation fails on format / structure"	Retry with specific error feedback
"Need to know which patterns produce noise"	Add `detected_pattern` field to schema for dismissal analysis
"Agentic workflow + batch?"	No — batch can't execute tools mid-request

08 Structured Output Tooling for CI

Domain 4 leans heavily on the mechanics of producing reliable structured output.

CLAUDE.md for CI Test Generation Quality

- When Claude Code runs in CI to generate tests, it reads CLAUDE.md for project context
- Higher-quality, more relevant tests result when CLAUDE.md includes:
  - Team's testing standards
  - Preferred assertion libraries
  - Available test fixtures
  - Criteria for valuable tests

EXAMPLE

Applied example

— Team uses Vitest with React Testing Library and shared fixtures in `/test/fixtures/`

— Without CLAUDE.md guidance: Claude generates Jest-syntax tests with inline mock data

— With the right CLAUDE.md entry: it uses Vitest, references existing fixtures, and focuses on integration tests rather than trivial unit tests for getters/setters

```
## Testing Standards
- Framework: Vitest + React Testing Library
- Shared fixtures: /test/fixtures/ (mock API responses, user objects)
- Valuable tests: Integration tests, error boundary tests, edge cases
- Low-value tests: Trivial getters/setters, snapshot tests of static content
- Always test error states alongside happy paths
```

TRAP

Exam trap

— A question may present CLAUDE.md as a distractor for output formatting (wrong) versus a correct answer for test quality context (right)

— CLAUDE.md provides project context and standards, NOT per-task output format instructions

`tool_use` with JSON Schemas for Guaranteed Structured Output

The pattern

- Don't prompt Claude to "respond in JSON format" — that can produce malformed JSON, markdown-wrapped JSON, or extra explanatory text
- Instead: define a tool with a JSON schema and force Claude to call it

What `tool_use` guarantees

- Output matches the schema structurally: correct types, required fields present, enums respected

What `tool_use` does NOT prevent

- Semantic errors (e.g., line items that don't sum to the stated total, or values placed in the wrong fields)

ERROR TYPE	DEFINITION	PREVENTED BY `TOOL_USE`?
Syntax errors	Malformed JSON, missing brackets, extra text outside JSON	YES — eliminated entirely
Structural errors	Wrong types, missing required fields, invalid enum values	YES — schema enforces structure
Semantic errors	Valid JSON but wrong values: amounts don't sum, data in wrong field	NO — requires validation-retry loop

EXAMPLE

Applied example

When structured output fails validation, don't retry blindly. Send a follow-up request that includes:

1. The original document/code
2. The failed extraction
3. The specific validation error

Claude uses the error message to self-correct. This is fundamentally different from simple retry. Section 3.5 covers when retries help versus when they don't.

EXAMPLE

Applied example in CI

- Claude extracts review findings from a PR diff
- Validation checks that every finding's file path actually exists in the PR's changed files
- Finding #3 references `src/utils/helpers.ts` but that file wasn't changed
- The retry prompt includes: "Finding #3 references `src/utils/helpers.ts`, but this file is not in the PR diff. The changed files are: [list]. Please re-extract finding #3 with the correct file path."

Batch Submission Frequency from SLA Constraints

- The Message Batches API can take up to 24 hours to process
- If the business has an SLA requiring results within a certain window, calculate how often to submit batches to guarantee meeting the SLA in the worst case

Formula

- Maximum submission interval = SLA deadline – Maximum batch processing time

EXAMPLE

Example

You've written a function. Now we're going to make it fail on purpose, and then fix it. This is how you learn to debug.